Lab No: 08

Lab Title: **Two-Dimensional Arrays**

- **Leaning objectives:**

The following topics will be covered in this lab session

- What is a Two-Dimensional Array
- When to use Two-Dimensional Array
- Declaration of Two-Dimensional array
- Initialization of Two-Dimensional Array
- Accessing Array Two-Dimensional element
- Copying Two-Dimensional arrays

The simplest form of the multidimensional array is the two-dimensional array. A two-dimensional array is, in essence, a list of one-dimensional arrays. To declare a two-dimensional integer array of size x,y, you would write something as follows −

```
type array Name [ x ][ y ];
```

Where **type** can be any valid C++ data type and **array Name** will be a valid C++ identifier.

A two-dimensional array can be thought as a table, which will have x number of rows and y number of columns. A 2-dimensional array **a**, which contains three rows and four columns can be shown as below

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Thus, every element in array a is identified by an element name of the form **a[i][j]**, where a is the name of the array, and i and j are the subscripts that uniquely identify each element in a.

Initializing Two-Dimensional Arrays:

Multidimensional arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row have 4 columns.

```
int a[3][4] = {
        {0, 1, 2, 3} ,   /*  initializers for row indexed by 0 */
        {4, 5, 6, 7} ,   /*  initializers for row indexed by 1 */
         {8, 9, 10, 11}   /* initializers for row indexed by 2 */
```

The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to previous example −

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

- **Accessing Two-Dimensional Array Elements**

An element in 2-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array. For example −

```
int val = a[2][3];
```

The above statement will take 4th element from the 3rd row of the array. You can verify it in the above diagram

```
#include <iostream>
using namespace std;

int main () {
  // an array with 5 rows and 2 columns.
  int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};

  // output each array element's value
  for ( int i = 0; i < 5; i++ )
    for ( int j = 0; j < 2; j++ ) {

      cout << "a[" << i << "][" << j << "]: ";
      cout << a[i][j]<< endl;
    }

  return 0;
}
```

- When the above code is compiled and executed, it produces the following result

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

- **Example no. 2:**

```
#include<iostream.h>
#include<conio.h>

main()
{

int matrix [5] [5];
for (int m1=0 ; m1<5 ; m1++)
{
for (int m2=0 ; m2<5 ; m2++)
{
matrix [m1] [m2] = 5 ;
}
}
getch();
}
```

- **Example no. 3:**

```
#include<iostream.h>
#include<conio.h>

main()
{
int matrix [2][3];
// For taking integer inputs in a matrix //
for (int m1=0 ; m1<2 ; m1++)
{
for (int m2=0 ; m2<3 ; m2++)
{
cout<<"Enter Integer :";
cin>>matrix [m1][m2];
}
}
cout<<endl;
 // For displaying elements of a matrix on a screen //
for (int m1=0 ; m1<2 ; m1++)
{
for (int m2=0 ; m2<3 ; m2++)
{
cout<<"Your Entered Integer are :";
cout<<matrix [m1][m2];
cout<<endl;
}
}
```

```
getch();
}
```

Lab Tasks

- **Task 1: Two-Dimensional Array Declaration and Initialization**

    1. Declare a 2D integer array called "matrix" with dimensions 3x3.
    2. Initialize the array with values 1, 2, 3, 4, 5, 6, 7, 8, and 9.
    3. Display the values of the array elements using nested loops.

Input:

```
#include <iostream>        // Input/Output library include kar rahe hain

using namespace std;       // Standard namespace use kar rahe hain

int main() {               // Program ka starting point

   // 3x3 ka 2D array declare aur initialize kar rahe hain
   int matrix[3][3] = {
      {1, 2, 3},      // Row 1
      {4, 5, 6},      // Row 2
      {7, 8, 9}       // Row 3
```

```
    };

    // Outer loop rows ke liye (i row ko represent karta hai)
    for(int i = 0; i < 3; i++) {

        // Inner loop columns ke liye (j column ko represent karta hai)
        for(int j = 0; j < 3; j++) {

            // Har element print kar rahe hain (row i, column j)
            cout << matrix[i][j] << " ";
        }

        // Har row ke baad next line par jaane ke liye
        cout << endl;
    }

    return 0;   // Program successful end
}
```

}**Output:**

```
1 2 3
4 5 6
7 8 9
```

- Task 2: Sum of Rows and Columns in a Two-Dimensional Array

    1. Declare and initialize a 3x3 integer array with random values.
    2. Calculate and display the sum of each row and each column separately.
    3. Display the total sum of all the elements in the array.

Input:

```
#include <iostream>     // Input/Output ke liye

using namespace std;

int main() {

   // 3x3 array initialize kar rahe hain
   int arr[3][3] = {
      {2, 4, 6},
      {1, 3, 5},
      {7, 8, 9}
   };

   int totalSum = 0;    // Total sum store karne ke liye variable

   // ===== Row Sum =====
   for(int i = 0; i < 3; i++) {     // Har row ke liye loop
      int rowSum = 0;              // Har row ka sum reset

      for(int j = 0; j < 3; j++) { // Row ke andar columns
         rowSum += arr[i][j];      // Row ke elements add kar rahe hain
         totalSum += arr[i][j];    // Total sum bhi update kar rahe hain
      }
```

```
        // Har row ka sum print
        cout << "Row " << i+1 << " Sum = " << rowSum << endl;
    }

    // ===== Column Sum =====
    for(int j = 0; j < 3; j++) {     // Har column ke liye loop
        int colSum = 0;              // Column sum reset

        for(int i = 0; i < 3; i++) { // Har row ka same column
            colSum += arr[i][j];     // Column ke elements add
        }

        // Har column ka sum print
        cout << "Column " << j+1 << " Sum = " << colSum << endl;
    }

    // Total sum print
    cout << "Total Sum = " << totalSum << endl;

    return 0;   // Program end
}
```

Output:

```
Sum of row 1 is :6
Sum of coloumn 1 is :12
Sum of row 2 is :15
Sum of coloumn 2 is :15
Sum of row 3 is :24
Total sum of all elements is:45
```

- **Task 3: Finding the Maximum Value in a Two-Dimensional Array**

    1. Declare and initialize a 4x4 integer array with random values.
    2. Find and display the maximum value in the array.

Input:

```
#include <iostream>    // Input/Output library

using namespace std;

int main() {

    // 4x4 array initialize kar rahe hain
    int arr[4][4] = {
        {10, 25, 3, 8},
        {14, 6, 19, 2},
        {7, 30, 11, 5},
        {9, 18, 22, 1}
    };

    // Pehle element ko maximum assume kar rahe hain
    int maxVal = arr[0][0];
```

```
    // Poora array traverse karne ke liye nested loops
    for(int i = 0; i < 4; i++) {        // Rows ke liye
        for(int j = 0; j < 4; j++) {    // Columns ke liye

            // Agar current element max se bara ho
            if(arr[i][j] > maxVal) {

                // To max value update kar do
                maxVal = arr[i][j];
            }
        }
    }

    // Final maximum value print
    cout << "Maximum Value = " << maxVal << endl;

    return 0;   // Program end
}
```

- **Output:Maximum value is 30**

Conclusion:

In this lab, we learned how to declare and initialize two-dimensional arrays in C++. We used nested loops to traverse the array and perform operations like displaying elements, calculating row and column sums, and finding the maximum value. This lab improved our understanding of array manipulation and logic building.